

សាកលវិទ្យាល័យ អាស៊ី អឺរ៉ុប

មហាវិទ្យាល័យ វិទ្យាសាស្ត្រ និងបច្ចេកវិទ្យា



មុខវិជ្ជា:

Programming Methodology in C++

មេរៀនទី៧:

ជំពូក ៧ Classes And Objects

ឆ្នាំទី ១ ឆមាស ទី ២

គណៈកម្មការអភិវឌ្ឍន៍កម្មវិធីសិក្សា

© 2024

មតិកា

- ❑ 1. ការបង្កើត និង ប្រើប្រាស់ Classes និង Objects
- ❑ 2. ការប្រើប្រាស់ Constructor និង Destructor
- ❑ 3. ការប្រើប្រាស់ Pointer to Class
- ❑ 4. ការប្រើប្រាស់ Class Defined with Structure , Union
- ❑ 5. ការប្រើប្រាស់ Overloading the Operators, Static Member
- ❑ 6. ការប្រើប្រាស់ FriendShip និង Inheritance
- ❑ 7. ការប្រើប្រាស់ Single និង Multiple Inheritance
- ❑ 8. ការប្រើប្រាស់ Pointer to Base Class Abstract Base Class

មតិកា

- ❑9. ការប្រើប្រាស់ Function Template និង Class Template
- ❑10. ការប្រើប្រាស់ Virtual Member , Template Specialization
- ❑11. ការប្រើប្រាស់ NameSpace និង ដោះស្រាយ Exception
- ❑ 12. ការប្រើ ការបំបែកប្រភេទទិន្នន័យ
- ❑១៣. ការកំណត់ Preprocessor Directives ការប្រើ Input/ output with Files.
- ❑14. ការប្រតិបត្តិរបៀប Read និង Write Files

1. ការបង្កើត និង ប្រើប្រាស់ Classes និង Objects

- 1.១. អ្វីទៅ ហៅថា Class ?
- ១.២. ការប្រើប្រាស់ Class .
- ១.3. អ្វីទៅ ហៅ ថា Objects ?
- 1.4. ការប្រើប្រាស់ Objects .
- 1.5. អ្វី ទៅ OOP ?
- 1.៦. OOP មានលក្ខណៈពិសេសៗ អ្វីខ្លះ?
- Example : Coding .

១.1. អ្វីទៅ ហៅថា Class ?

- **Class** គឺជាថ្នាក់រៀនមែនទេ? មិនមែន! ប៉ុន្តែវាអាចជាប្រភេទ ឬក៏នៃទិន្នន័យអ្វីមួយទៅវិញទេ ឬអាចនិយាយថា ជា ពុម្ពគំរូមួយ ឬក៏ជា សំណុំ។ class គឺជាគំរូនៃទិន្នន័យអ្វីមួយ ដែលអាចហៅថាជា data type ឬ user-defined types ដ៏ពិសេសមួយ ដែលត្រូវបានអ្នក programmers បង្កើតឡើងផ្សេងៗគ្នាសម្រាប់ការងារតំណាងទិន្នន័យអ្វីមួយនៅក្នុង programs របស់ពួកគេ ។
- ឧទាហរណ៍ ៖ មនុស្ស, សត្វ , វត្ថុ។

1.2. ការប្រើប្រាស់ Class

- នៅក្នុងភាសា C++ អ្នកអាចបង្កើត (definition) class មួយ ឬច្រើន តាមតម្រូវការ ជាក់ស្តែងនៅក្នុង programs របស់អ្នកដោយមិនកំណត់ចំនួនឡើយ ។ ការបង្កើត class ត្រូវអនុលោមតាមទម្រង់ដូចខាងក្រោម៖

- **Syntax :**

```
class ClassName {  
  
    members  
  
};
```

1.2. ការប្រើប្រាស់ Class (ត)

- **members** មានចំនួនពីរប្រភេទ គឺ data members ឬហៅថា member attributes ឬក៏ជា member variables និង methods ឬហៅថា member functions ។
- - data members គឺសំដៅលើ variables ដែលបង្កើតនៅក្នុង block របស់ class
- - methods គឺសំដៅលើ functions ដែលបង្កើតនៅក្នុង block របស់ class data ឬ methods ទាំងពីរប្រភេទនេះមិនដាច់ខាតត្រូវតែមានទាំងពីរនោះទេ គឺអាស្រ័យតាមតម្រូវការនៃ class ដែលត្រូវបង្កើត ។
- Class មួយ អាចបង្កើត object បានច្រើន គ្រាន់តែ object នោះត្រូវឈ្មោះខុសៗគ្នា, Object មួយស្ទើរ បណ្តាំ ពិតៗមានមួយ ។

១.3. អ្វីទៅ ហៅ ថា Objects ?

- **Objects** មានន័យថា អ្វីៗនៅជុំវិញខ្លួន យើង ។ គ្រប់ Object ត្រូវមានរូបរាង

(attributes) និង សកម្មភាព (behaviour)។ **Objects** ជាបស់ ដែលកើតចេញពី Class ហើយមានលិទ្ធភាព ប្រើបស់មួយចំនួន ក្នុង Class ដែល គេអនុញ្ញាត។

1.4. ការប្រើប្រាស់ Objects

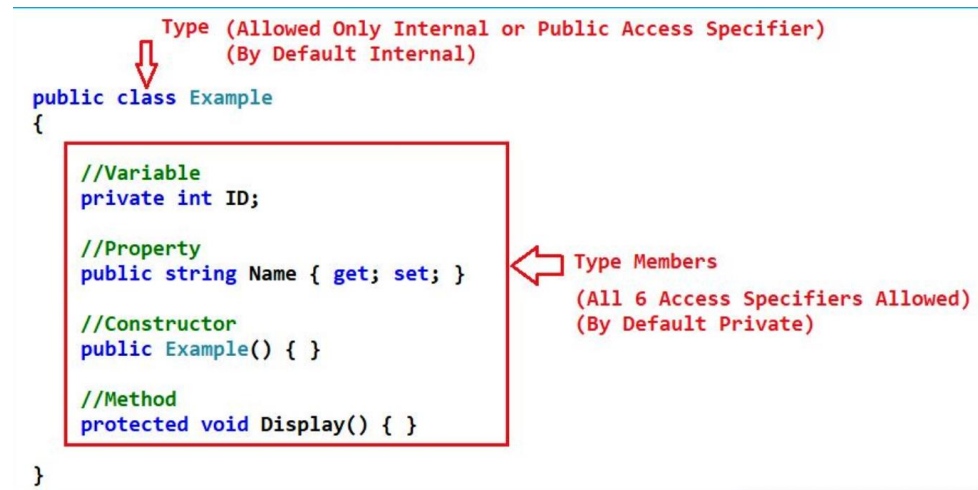
❖ Syntax :

ClassName ObjectName

ឧទាហរណ៍ ៖ , សិក្សា,ញ៉ាំ, ស, ខ្មៅ, តំបន់ ,ភេទ, ឈ្មោះ , អត្តលេខ, វែង ,
ខ្លី ,កៅអី,អាយុ...។

Access Specifiers

- **Access Modifiers (ឬ access specifiers)** គឺជាពាក្យគន្លឹះ:(Keyword) នៅក្នុង OOP ដែលកំណត់ភាពអាចចូលទៅដល់នៃ Class, Method និង Member ផ្សេងទៀត។ នៅក្នុងភាសា C++ មាន Access Modifiers ចំនួន ៣ គឺ: Private, Public និង Protected។



```

Type (Allowed Only Internal or Public Access Specifier)
(By Default Internal)
↓
public class Example
{
    //Variable
    private int ID;

    //Property
    public string Name { get; set; }

    //Constructor
    public Example() { }

    //Method
    protected void Display() { }
}
  
```

Type Members
 (All 6 Access Specifiers Allowed)
 (By Default Private)

Access Specifiers(ត)

- ពាក្យគន្លឹះ **Public** គឺជា Access modifiers មួយក្នុងចំណោមដែលមានបីនោះ ។ Access Specifiers កំណត់ពីរបៀបដែល Member(attributes និង methods) នៃ Class អាចចូលប្រើបាន ។ នៅក្នុងឧទាហរណ៍ខាងលើ Member គឺ public ដែលមានន័យថាពួកគេអាចហៅទៅប្រើគ្រប់ទីកន្លែងនៃកូដ។
 - ❖ Public – member អាចចូល access បានពីខាងក្រៅថ្នាក់។
 - ❖ Private – member មិនអាចចូល access ពីខាងក្រៅថ្នាក់បានទេ ។
 - ❖ Protected – member មិនអាចចូល access ពីខាងក្រៅថ្នាក់បានទេ ។ ទោះយ៉ាងណាក៏ដោយ ពួកគេអាចចូល access បានក្នុង Class ដែលទទួលមរតក ។ អ្នកនឹង ស្វែងយល់បន្ថែមអំពី Inheritance នៅពេលក្រោយ។

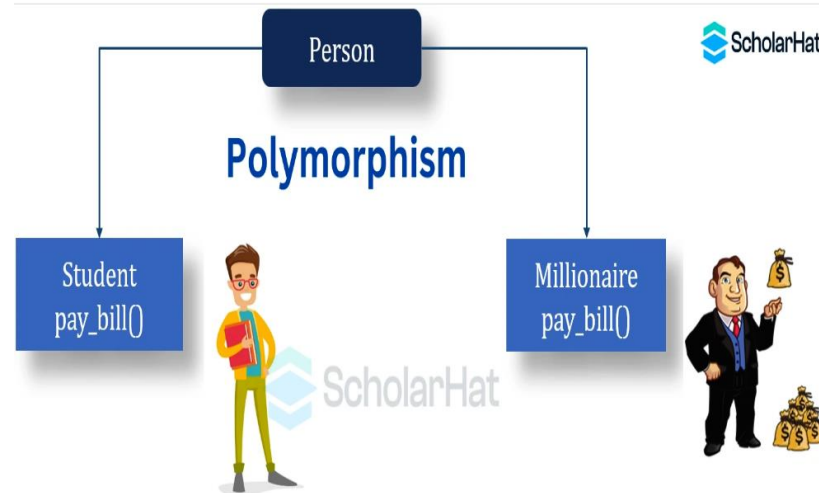
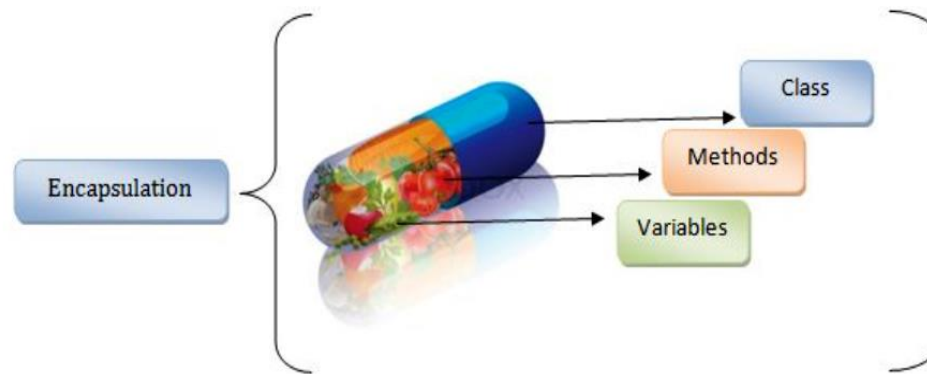
1.5. អ្វី ទៅ OOP?

- **OOP** មានពាក្យពេញថា **Object Oriented Programming** គឺជាវិធីសាស្ត្រដ៏មានសារៈសំខាន់ក្នុងការសរសេរកម្មវិធីកុំព្យូទ័រ ដោយការប្រើប្រាស់ **Classes** or **Object** ដែលចងភ្ជាប់ក្នុងជាមួយគ្នា ដោយមិនចាំបាច់ក្នុងការសរសេរក្នុងដដែលៗ និងដំណើរការប្រតិបត្តិការយ៉ាងសាមញ្ញ ។ **OOP** មានដំណើរការលឿន សាមញ្ញ និងមានភាពងាយស្រួលក្នុងការដោះស្រាយបញ្ហា ប្រើប្រាស់ធនធាន **Server** តិច និងការសរសេរក្នុងតិច ដំណើរការលឿននិងត្រឹមត្រូវជាងមុនក្នុងការធ្វើការនៅពេលដែលអ្នកដឹងពីមូលដ្ឋានគ្រឹះរបស់វា។
- **OOP** ត្រូវបានប្រើក្នុងភាសា programming ពេញនិយមជាច្រើនដូចជា៖ **Java, C++, C#, Visual Basic.NET, Python, JavaScript** និង **PHP** ។

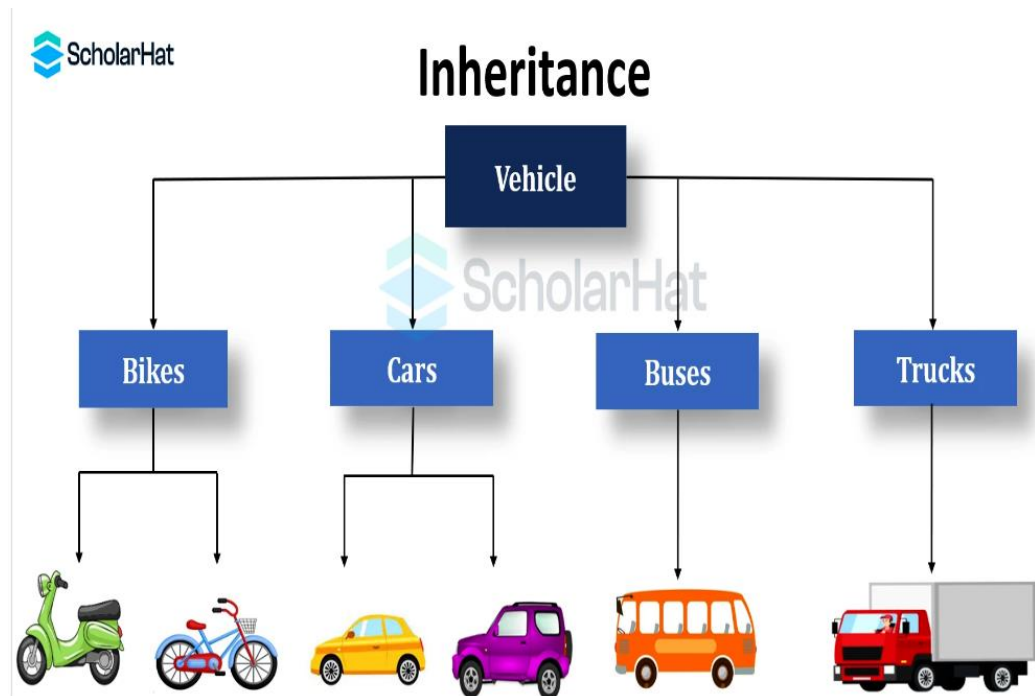
១.៦. OOP មានលក្ខណៈពិសេសៗមួយចំនួនដូចខាងក្រោម ៖

- **Encapsulation** ៖ ការលាក់ភាពសម្បត្តិទិន្នន័យនិងប្រតិបត្តិការខាងក្រោយកម្មវិធី។
- **Polymorphism** ៖ ការប្រើឈ្មោះឬ សញ្ញាដូចគ្នាបាន (ក្នុង **Class** មានឈ្មោះ **Variable** នោះហើយ ប៉ុន្តែនៅក្នុង **Class** ផ្សេងក៏អាចប្រើឈ្មោះនោះបានដែរ), ការបង្កើតនូវអ្វីមួយដោយអាចយកទៅប្រើបានច្រើនទិសដៅ ក្នុងនោះមាន **Overloading method** ជាដើម។
- **Inheritance** ៖ ការផ្ទេរមរតក (**Object** អាចទទួលបានតាមការកំណត់របស់ **Class**) ។ **Data members, Properties, or Method**: ដែលប្រកាសជា **Public or Protected** អាចបន្តមរតក។ **Private Members** មិនអាចបន្តមរតកបានទេ។

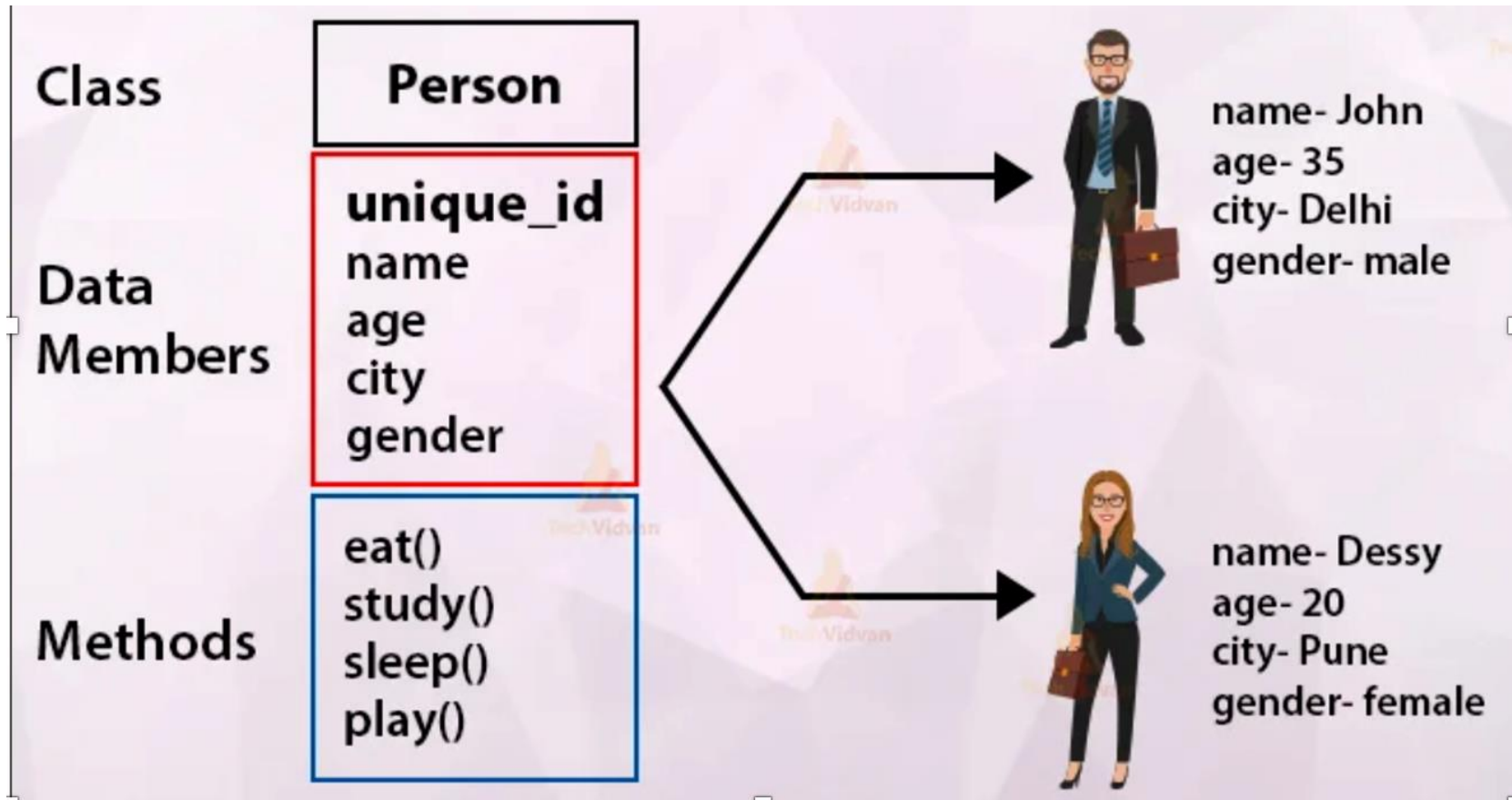
រូបភាពតំណាង



រូបភាពតំណាង



រូបភាពតំណាង Class និង Objects



Coding

• Example : Coding Class and Object

```
#include <iostream>
using namespace std;
class Test{
public:
    int x ;
    int y;
};
int main(){
    Test t;
    cout<<"Input X = "; cin>>t.x;
    cout<<"Input Y = "; cin >> t.y;
    cout<<"X="<<t.x<<endl;
    cout<<"Y = " <<t.y<<endl;
    cout<<"X+Y = " <<t.x + t.y<<endl;
    cout<<"X-Y = " <<t.x - t.y <<endl;
    cout<<"X * Y = " << t.x * t.y <<endl;
    cout<<"X/Y = " << t.x * t.y << endl;
    return 0;
}
```

```
Input X = 100
Input Y = 83
X=100
Y = 83
X+Y = 183
X-Y = 17
X * Y = 8300
X/Y = 8300
```

=== Code Execution Successful ===



Coding

Example : Coding Class and Object

```

2  #include <iostream>
3  using namespace std;
4  class Employee{
5      //data member
6      private:
7          int id;
8          string name ;
9          char gender;
10         float salary;
11     public :
12         // function member
13         void Input()
14     {
15         cout<<"Input ID="<<cin>>id;
16         cout<<"Input Name = "; cin>>name;
17         cout<<"input Gender= "; cin>> gender ;
18         cout<<"Input Salary = "; cin>> salary;
19     }
20     // function member
21     void Output()
22     {
23         cout<< "ID = "<< id<< endl;
24         cout<< "Name = " << name << endl;
25         cout<< "Gender = " << gender << endl;
26         cout<< "Salary = " <<salary << endl;
27     }
28 };
29
30 int main(){
31     //object
32     Employee emp1,emp2;
33     emp1.Input();
34     emp1.Output();
35     return 0;
36 }

```

```

Input ID=2
Input Name = malen
input Gender= female
Input Salary = ID = 2
Name = malen
Gender = f
Salary = 0

```

=== Code Execution Successful ===



Coding Inheritance

<pre> 1 // Online C++ compiler to run C++ program online 2 #include <iostream> 3 #include <string> 4 using namespace std; 5 //Base class 6 class Vehicle { 7 //access specifier(public) 8 public: 9 // data type(string) variable(brand) 10 string brand = "Toyota"; 11 //method or function 12 void color(){ 13 cout << "Color : red\n" ; 14 } 15 }; 16 // inheritance class vehicle 17 class Car : public Vehicle{ 18 public: 19 string model = "Japen"; 20 }; 21 22 int main(){ 23 // cread object (myCar) 24 Car myCar; 25 myCar.color(); 26 cout << "Brand : " + myCar.brand + "\n" + 27 "Model : " + myCar.model; 28 return 0; </pre>	<pre> /tmp/tQSfGYbdCa.o Color : red Brand : Toyota Model : Japen === Code Execution Successful === </pre>
--	--

Coding Polymorphism

```
#include <iostream>
#include <string>
using namespace std;

// Base class
class Animal {
public:
    void animalSound() {
        cout << "The animal makes a sound \n";
    }
};

// Derived class
class Pig : public Animal {
public:
    void animalSound() {
        cout << "The pig says: wee wee \n";
    }
};

// Derived class
class Dog : public Animal {
public:
    void animalSound() {
        cout << "The dog says: bow wow \n";
    }
};

int main() {
    Animal myAnimal;
    Pig myPig;
    Dog myDog;

    myAnimal.animalSound();
    myPig.animalSound();
    myDog.animalSound();
    return 0;
}
```

The animal makes a sound
The pig says: wee wee
The dog says: bow wow

=== Code Execution Successful ===



Coding Encapsulation

```
#include <iostream>
using namespace std;

class Employee {
private:
    // Private attribute
    int salary;

public:
    // Setter
    void setSalary(int s) {
        salary = s;
    }
    // Getter
    int getSalary() {
        return salary;
    }
};

int main() {
    Employee myObj;
    myObj.setSalary(50000);
    cout << myObj.getSalary();
    return 0;
}
```



សាកលវិទ្យាល័យ អាស៊ី អឺរ៉ុប

ASIA EURO UNIVERSITY

សិក្សាស្រាវជ្រាវ គ្រប់ទីកន្លែង

អរគុណ

សម្រាប់ការយកចិត្តទុកដាក់

<https://elearning.aeu.cloud>